Let's say you want to change your project view so behind-schedule projects show up differently. You could do that in a helper. My normal practice is to add a CSS class to the output for both the regular and behind-schedule cases, to give the design maximum freedom to display as desired.

Here's a test for that helper:

display/01/spec/helpers/projects_helper_spec.rb

```
Line 1  require "rails_helper"
-
-  RSpec.describe ProjectsHelper, type: :helper do
-    let(:project) { Project.new(name: "Project Runway") }
5
-    it "augments with status info when on schedule" do
-      allow(project).to receive(:on_schedule?).and_return(true)
-      actual = helper.name_with_status(project)
-      expect(actual).to have_selector("span.on_schedule", text: "Project Runway")
10   end
-  end
```

In this test you're creating a new project using a standard ActiveRecord new method. Rather than define a bunch of tasks that would mean the new project is on schedule, you just stub the on_schedule? method on line 7 to return true. This has the advantage of being faster than creating a bunch of objects and, I think, being more clear as to the exact state of the project being tested. You're using the have_selector matcher, as you did in Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151, to compare the expected HTML with the generated HTML.

That test will fail, however, because you haven't defined the name_with_status helper. Let's define one:

display/01/app/helpers/projects_helper.rb

```
module ProjectsHelper
  def name_with_status(project)
    content_tag(:span, project.name, class: "on_schedule")
  end
end
```

The test passes. Now let's add a second test for the remaining case. This test will look familiar:

display/02/spec/helpers/projects_helper_spec.rb

```
it "augments with status info when behind schedule" do
  allow(project).to receive(:on_schedule?).and_return(false)
  actual = helper.name_with_status(project)
  expect(actual).to have_selector(
    "span.behind_schedule", text: "Project Runway")
end
```